



REFERENCE MANUAL

Feature atlas, legacy shell, testing, plugin examples

Open charting PWA — own the axes, swap the engine



CONTENTS

01	Feature Atlas
02	Open Capability Gaps
03	Legacy Shell
04	Testing
05	Plugin Examples

FEATURE ATLAS

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED

THIS FILE IS PART OF PYNESCRYPT.

PYNESCRYPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY

IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY

THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR

(AT YOUR OPTION) ANY LATER VERSION.

PYNESCRYPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL, BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF

MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.

YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE

ALONG WITH PYNESCRYPT. IF NOT, SEE

[HTTPS://WWW.GNU.ORG/LICENSES/](https://www.gnu.org/licenses/).

SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Feature atlas" description: "Map of AXIS product surfaces to repository paths: plugins, AXIS, worker, storage, test, legacy."

FEATURE ATLAS

Abstract

This atlas is a **path index** for AXIS. Use it to jump from a product feature to the code that implements it. Prefer Solid product paths over legacy static shell files.



Conceptual model

```
mindmap
  root((AXIS))
    Plugins
      sources
      streams
      engines
      storage
      loader
    AXIS
      chart
      editor
      ui
      store
    Worker
      run
      keys
      scripts
      DO
    DevOps
      vite
      pages
      ci
```

Atlas tables

Plugin system

Feature	Primary paths
Contracts	src/plugins/types.ts
Registry	src/plugins/registry.ts
Bootstrap	src/plugins/bootstrap.ts
Active set	src/plugins/active.ts
Dynamic install	src/plugins/loader.ts (source/stream/engine/dataset/ component)
Barrel	src/plugins/index.ts
Manager UI	src/ui/PluginManager.tsx , plugin-badges.tsx
Docs (in-tree)	src/plugins/README.md



Data plugins

Feature	Primary paths
Sources	<code>src/sources/catalog.ts</code> , <code>upload-store.ts</code>
Streams	<code>src/streams/catalog.ts</code> , <code>multiplex.ts</code>
Engines	<code>src/engines/catalog.ts</code>
Load symbol (one-shot)	<code>src/data/load-symbol.ts</code> , <code>parse-bars.ts</code>
Data Source Manager	<code>src/data/data-source-manager.ts</code> , <code>bars-cache.ts</code> , <code>bars-gaps.ts</code> , <code>ui/DataSourceManagerPanel.tsx</code>
Script run	<code>src/indicators/runner.ts</code>
Scripts panel	<code>src/indicators/IndicatorPanel.tsx</code> , <code>IndicatorCard.tsx</code>
Built-in script catalog	<code>src/indicators/builtins/</code>
Chart themes	<code>src/theme/presets.ts</code> , <code>catalog.ts</code> , <code>manager.ts</code> , <code>ui/ThemePanel.tsx</code>

Storage / library

Feature	Primary paths
Catalog	<code>src/storage/catalog.ts</code>
Local IDB	<code>src/storage/local.ts</code> , <code>idb.ts</code>
Cloud	<code>src/storage/cloud.ts</code>
Git	<code>src/storage/git.ts</code> , <code>git-github.ts</code> , <code>git-gitlab.ts</code>
Service façade	<code>src/storage/service.ts</code>
Library panel	<code>src/ui/ScriptLibraryPanel.tsx</code>
v6 starter pack	<code>examples/script-library-starter.json</code>
First-party built-in scripts	<code>src/indicators/builtins/</code>



UI (UI)

Feature	Primary paths
Entry	<code>src/index.tsx</code> , <code>src/app.tsx</code>
Chart host	<code>src/chart/ChartHost.tsx</code> , <code>ChartWorkspace.tsx</code> , <code>pane-manager.ts</code> , <code>pane-badge.ts</code> , <code>series-factory.ts</code>
Price-scale decimals	<code>src/chart/price-precision.ts</code>
Plot style parity	<code>src/results/plot-visuals.ts</code> → <code>pane-manager</code>
Multi-chart / recipes	<code>src/chart/layout.ts</code> , <code>layout-recipes.ts</code> , <code>chart-registry.ts</code> , <code>ui/ChartLayoutMenu.tsx</code>
Bar replay	<code>src/chart/bar-replay.ts</code> , <code>ui/BarReplayControls.tsx</code>
Compare / volume profile	<code>src/chart/compare-overlay.ts</code> , <code>volume-profile.ts</code>
Drawings	<code>src/chart/drawing-layer.ts</code> , <code>drawings/*</code> , <code>pyne-drawings.ts</code> , <code>DrawingToolbar.tsx</code>
Editor	<code>src/editor/*</code> (CM6, diagnostics, ruler, git-sync, inline-debug, symbols)
Editor chrome	<code>ui/EditorGitBar.tsx</code> , <code>ui/EditorProblems.tsx</code>
Alerts	<code>src/alerts/*</code> , <code>ui/AlertsPanel.tsx</code>
Command palette	<code>ui/CommandPalette.tsx</code> , <code>command-registry.ts</code>
Docks	<code>ui/panels/*</code> (side-by-side left/right)
Results	<code>src/results/*</code> , <code>ui/ResultsPanel.tsx</code> , <code>StrategyReport.tsx</code>
Workers Manager	<code>src/ui/WorkersManager.tsx</code> , <code>src/workers/*</code>
Store	<code>src/store/index.ts</code> , <code>types.ts</code>
Topbar / settings	<code>ui/Topbar.tsx</code> , <code>TopbarField.tsx</code> , <code>SettingsDialog.tsx</code>
Workspace snapshot	<code>src/storage/workspace-snapshot.ts</code>
Desktop shell bridge	<code>src/desktop/*</code> , <code>src-tauri/</code>



Worker

Feature	Primary paths
Router / CORS	<code>worker/src/index.ts</code> (<code>pickOrigin</code>)
Run (auth + rate limit)	<code>worker/src/runtime.ts</code> , <code>pyodide_runtime.ts</code>
Auth / keys	<code>worker/src/auth.ts</code> , <code>keys.ts</code>
Scripts	<code>worker/src/scripts.ts</code> , <code>schemas/scripts.sql</code>
Git OAuth	<code>worker/src/git-oauth.ts</code>
Session DO	<code>worker/src/durable-objects/session.ts</code>
Config	<code>worker/wrangler.toml.example</code> , <code>RUNTIME.md</code> , <code>README.md</code>
MCP server	<code>worker/src/mcp/</code> (JSON-RPC <code>/mcp</code> + <code>McpBridgeDO</code>)
MCP PWA host	<code>src/mcp/</code> (capabilities, snapshot, WS bridge)
MCP CLI	<code>packages/cli/src/commands/mcp.ts</code> (<code>axis mcp</code>)

Build / ops

Feature	Primary paths
Package scripts	root <code>package.json</code> (<code>dev</code> , <code>build</code> , <code>test:*</code> , <code>axis:*</code> , <code>desktop:*</code>)
AXIS CLI	<code>packages/cli/</code> (<code>@hoox-sh/axis-cli</code>)
Static server	<code>axis_pwa_server.py</code>
Public assets	<code>public/plugins</code> , <code>public/vendor</code> , <code>public/pyodide</code>
Makefile	<code>axis</code> / <code>axis-*</code> , <code>pages-deploy</code> , <code>docker-*</code>
Desktop CI	<code>.github/workflows/desktop.yml</code>
CI	<code>.github/workflows/ci.yml</code> (coverage + e2e smoke)
Coverage gate	<code>scripts/check-coverage.mjs</code> (scoped core $\geq 95\%$)
Harden/perf audit	<code>docs/devops/harden-perf-audit-2026-08-11.md</code>

Examples & tests

Feature	Primary paths
Example plugins	<code>public/plugins/*.js</code>
Pine v6 library starter	<code>examples/script-library-starter.json</code>
First-party built-in Pine	<code>src/indicators/builtins/</code>
Unit tests	<code>tests/**</code>
Worker tests	<code>worker/tests/**</code>
E2E	<code>e2e/**</code> , <code>playwright.config.ts</code>
Testing guide	<code>TESTING.md</code>
Legacy notes	<code>LEGACY.md</code>



Namespaces & keys

Key / namespace	Use
<code>pynescrypt.axis.plugins.v1</code>	Installed dynamic plugins
<code>pynescrypt.axis.library.v1</code>	Local library LS fallback
<code>pynescrypt.axis.storage</code>	IDB database name
<code>pynescrypt.axis.v1</code>	App state
<code>pynescrypt.axis.v2</code>	Older app-state key (write-forward)
Contract ns	<code>pynescrypt.axis.plugins.v1</code> (conceptual)

Infrastructure constants

Name	Value
Pages project	<code>axis</code>
Pages URL	<code>https://axis.hoox.sh</code>
Worker script	<code>worker-axis</code>
Worker URL	<code>https://worker.axis.hoox.sh</code>
Health service	<code>worker-axis</code>
Pyodide version	<code>0.26.2</code> (self-hosted path)

See also

- [Plugins](#)
- [Worker](#)
- [Legacy shell](#)

OPEN CAPABILITY GAPS

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED

THIS FILE IS PART OF PYNESCRYPT.

PYNESCRYPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY

IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY

THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR

(AT YOUR OPTION) ANY LATER VERSION.

PYNESCRYPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL, BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF

MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.

YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE

ALONG WITH PYNESCRYPT. IF NOT, SEE

[HTTPS://WWW.GNU.ORG/LICENSES/](https://www.gnu.org/licenses/).

SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Open capability gaps" description: "Research synthesis of high-demand charting, strategy, and script-host capabilities that remain scarce or incomplete in mainstream platforms—framed for AXIS product direction." sidebarTitle: "Capability gaps"

OPEN CAPABILITY GAPS

Abstract

This page records a **product research synthesis**: roughly thirty concrete capabilities that traders, quant authors, and chart power-users still treat as missing or incomplete on mainstream charting and strategy hosts. It is not a shipped-feature checklist for AXIS and not a competitive teardown of any named vendor.



Use it when prioritizing AXIS surfaces (sources, streams, engines, strategy results, debugging, data plane). Items below are durable demand themes from community threads and documented hard limits—not a public vendor roadmap with delivery status.

How to read this list

Term	Meaning here
Still open	Capability is absent, plan-gated to a partial substitute, or still described as unreliable for professional use
Not a ranking poll	No single global leaderboard exists; order is thematic, then by how often demand shows up as trade-blocking
Status changes	Individual venues or UX affordances can ship later; verify against current product notes before treating any row as forever

Highest-pressure themes overall: true tick and order-flow data, portfolio-grade strategy tooling, live multi-bracket and automated execution, higher cloud/screener ceilings, and a real interactive debugger.

Order flow, volume, and tick data

#	Capability	Why it stays open
1	True tick volume under volume profiles (including reliable delta)	Profiles over the same range can disagree on buy/sell aggression; many treat minute-aggregated volume as unsuitable for professional volume work
2	Real volume footprint with aggressor-tagged prints	Synthetic or approximate footprints are still not trusted next to tick-native tools
3	Transaction-level order-flow viewing	Charts that are primarily bar/minute-based cannot show trade-by-trade flow without true tick history

Live trading, brackets, and execution

#	Capability	Why it stays open
4	Multi take-profit / multi-bracket management on live broker routes	Multi-exit levels often exist in paper/simulators while live multi-exit brackets remain incomplete or broker-dependent
5	Direct automated live order placement from strategy scripts	Strategy engines commonly fill via a broker <i>emulator</i> ; real exchange orders need external webhooks or bridges
6	Unified routing of strategy orders through the same trading panel path as manual/paper orders	Script strategies and discretionary trading panels are often separate stacks



Strategy model: portfolio, hedge, and fill realism

#	Capability	Why it stays open
7	Native multi-symbol / portfolio strategy backtesting	One script run is typically bound to one chart dataset; multi-asset P&L is export-and-compare, not a portfolio engine
8	Simultaneous long + short (hedge) on the same symbol	Position models are often single-direction at a time
9	Positions in symbols other than the chart asset	Cross-asset entries from one strategy remain unsupported on many hosts
10	Full tick-level historical fill realism	Fills usually walk chart OHLC with assumed open → high → low → close (or open → low → high → close) paths; higher-resolution path reconstruction is partial and often tier-gated
11	Large trade histories without silent trim	Non-deep backtests often cap order history (older trades dropped from testers); deep modes raise the ceiling but remain product-tier features

Script runtime, quotas, and tooling

#	Capability	Why it stays open
12	Higher unique multi-series / remote-series request ceilings	Cloud hosts hard-cap distinct series fetches per script (plan tiers only raise the number)
13	Higher total script wall-time budgets	Entire-run timeouts remain fixed per plan class
14	Higher per-bar loop budgets	Tight loop-per-bar limits reject heavy iterative logic
15	Higher data, memory, and compiled-size caps	Large libraries and data-heavy studies hit opaque resource walls
16	Large multi-symbol scanning without burning unique-request quotas	Dynamic loops still count each distinct symbol/timeframe against the unique ceiling
17	Runtime profilers (wall time, memory, compiled size) before failure	Authors discover limits by rejection or runtime error, not by measurement tools
18	Interactive step-through debugger with breakpoints	Official debugging remains logs, plots, drawings, and chart colors—not a classic debugger

Script-driven screener constraints

#	Capability	Why it stays open
19	Multiple custom indicators per screener pass	Many screeners allow only one user script per screen
20	Indicator-on-indicator composition inside the screener	Nested studies are unsupported
21	Custom timeframes in scripted screens	Non-standard bars often unavailable
22	Higher remote-series call counts inside screener scripts	Typical caps are very small (single-digit)
23	Lookback beyond a short trailing window (e.g. hundreds of bars only)	Screen math is truncated to recent history
24	Full input-type coverage for screener-hosted scripts	Several input kinds are unsupported in the screener surface



Chart UX and market coverage

#	Capability	Why it stays open
25	Stable, discoverable watchlist gestures (e.g. context-menu add)	Removal or relocation of common chart chrome produces sustained restore demand
26	Timely exchange / venue chart coverage when a market is popular	Coverage gaps surface as high-engagement “still missing” demand until a data feed lands
27	Sector / industry / theme heatmaps and similar multi-name overview boards	Recurring UX requests outside pure charting

Open script host vs general-purpose stacks

#	Capability	Why it stays open
28	External data sources, HTTP, and host APIs from inside scripts	Cloud sandboxes intentionally block outbound and arbitrary I/O
29	Databases, file I/O, and machine-learning libraries inside the script runtime	Language surface is chart-bound, not a general quant stack
30	True custom UI and host programming beyond chart plots and fixed panels	Outgrow path is usually a multi-asset engine with live broker APIs outside the chart host

Themes for AXIS

These gaps map cleanly onto AXIS axes without implying any third-party product parity:

Demand theme	AXIS lever
Tick / order-flow fidelity	Pluggable sources and streams with higher-resolution bars or trades when data exists
Portfolio multi-symbol strategies	Multi-source strategy evaluation and results beyond single-chart P&L
Live multi-exit and automation	Optional execution adapters / webhooks (explicit, user-owned—not silent live routing)
Runtime ceilings and profilers	Local and Worker engines with measurable limits you control
Interactive debugger	Step and pin tooling against the active engine (see UI debugging surfaces)
Screener scale	Offline or self-hosted scan loops unbound by a single vendor’s cloud quota
External data / ML / DB	Host bridges and plugins—not inventing closed chart-host APIs

AXIS remains a composition host: sources load history, streams advance the present, engines evaluate scripts, storage holds libraries. Capability work should preserve that separation.

Related docs

- [Architecture overview](#) — composition model
- [Strategy and results \(end user\)](#) — how strategy output is surfaced today
- [Debugging \(UI\)](#) — current debug surfaces
- [Feature atlas](#) — path index for implementation work



Research notes

- Synthesis date context: community and documentation signals through early–mid 2026.
- “Still open” means encoded as present limits or still treated as incomplete in high-engagement demand—not items already reversed by later product notes.
- Engagement is qualitative across forums and social posts; it is not a weighted global survey.
- Plan-gated historical depth and deep-backtest order ceilings partially address older “more bars / more trades” asks; those are de-emphasized relative to portfolio, execution, order-flow, and tooling gaps.

LEGACY SHELL

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED

THIS FILE IS PART OF PYNESCRYPT.

PYNESCRYPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY

IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY

THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR

(AT YOUR OPTION) ANY LATER VERSION.

PYNESCRYPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL, BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF

MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.

YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE

ALONG WITH PYNESCRYPT. IF NOT, SEE

[HTTPS://WWW.GNU.ORG/LICENSES/](https://www.gnu.org/licenses/).

SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Legacy shell" description: "Pre-Solid static shell vs current AXIS product path — what to ignore and what still runs."

LEGACY SHELL

Abstract

AXIS's shipping product path is **Solid + Vite**. An older **static shell** remains in the repo for historical scripts and some Bun tests. Do not document or extend the legacy shell as the primary UI.

Source of truth: `LEGACY.md`.



Conceptual model

```
flowchart LR
    LEG[Legacy main.js tree]
    CUR[Solid index.tsx]
    LEG -->|keep for tests| BUN[bun test static bits]
    CUR --> SHIP[dist / Pages]
```

What is legacy

Path	Role
src/main.js	Old bootstrap (chart.js, legacy-topbar.js, ...)
style.css	TV-blue-era tokens
pyne-editor.js	Pre-Solid CM6 wiring (renamed from pine-editor.js)
server.ts	Bun static server for old tree
sw.js	Service worker for the old shell

There is no `frontend/` directory and no `make run-frontend` target. The product path is `bun run dev`. A static demo of `dist/` is `python axis_pwa_server.py` (default port 8081) after `bun run build`.

What is current

Path	Role
src/index.tsx, app.tsx	Solid app
src/chart/ChartHost.tsx	LWC panes
src/ui/*	Topbar, Settings, Results, Manager, ...
public/ + bun run build	Production PWA assets
axis_pwa_server.py	Serve dist for demos

Migration residue

Residue	Handling
pynescrypt.axis.plugins.v1	Plugin install list
pynescrypt.axis.library.v1	Library localStorage fallback
pynescrypt.axis.editor.doc	Editor document backup
CF Pages project axis	Pages id; canonical <code>https://axis.hoox.sh</code>
CF Worker worker-axis	Script name; URL <code>https://worker.axis.hoox.sh</code>
Some docs/Makefile strings	Current brand is AXIS

App state namespace: `pynescrypt.axis.v1`.



Invariants

- 1. Do not port new features into `main.js` .
- 2. Prefer Solid store (`src/store`) over `state.js` .
- 3. Plugin registry is the Solid path (`src/plugins/registry.ts`); dual registries in old JS should be treated as dead ends.
- 4. Pages deploy should use Vite `dist/` , not the repo root static tree.

Failure modes

Confusion	Clarification
“UI doesn’t match docs”	You may be serving the legacy shell
Plugin manager missing	Legacy topbar lacks Solid Manager
Tests pass, UI wrong	Unit tests may still import legacy helpers

See also

- [Feature atlas](#)
- [Build and serve](#)
- [AXIS hub](#)

TESTING

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED

THIS FILE IS PART OF PYNESCRYPT.

PYNESCRYPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY

IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY

THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR

(AT YOUR OPTION) ANY LATER VERSION.

PYNESCRYPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL, BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF

MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.

YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE

ALONG WITH PYNESCRYPT. IF NOT, SEE

[HTTPS://WWW.GNU.ORG/LICENSES/](https://www.gnu.org/licenses/).

SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Testing reference" description: "AXIS unit, worker, security, coverage gate, and Playwright e2e conventions with paths and commands."

TESTING REFERENCE

Abstract

AXIS tests are **Bun-first** under the repo root, with Playwright for browser smoke and a scoped coverage ratchet. Canonical human guide: `TESTING.md`. This page is the reference map for agents and contributors.



Conceptual model

```
flowchart TB
    U[Unit Bun] --> CORE[plugins storage store catalogs]
    W[Worker Bun] --> API[auth keys scripts runtime]
    S[Security] --> THREAT[URL schemes partitions If-Match]
    E[Playwright] --> SMOKE[@smoke PR]
    E --> FULL[nightly all]
    C[Coverage gate] --> SCOPED[95% scoped lines]
```

Commands

```
bun run test          # unit + worker tests
bun run test:unit     # tests only
bun run test:coverage # lcov + text → coverage/
bun run test:coverage:gate # coverage + 95% scoped gate
bun run test:security # tests/security/
bun run test:e2e:smoke # Playwright @smoke
bun run test:e2e:critical # @smoke|@critical
bun run test:e2e      # all
bun run test:all      # gate + security
```

Layout

```
tests/
  setup.ts
  fixtures/
  helpers/
  *.test.ts
  integration/
  security/
scripts/check-coverage.mjs
e2e/
  smoke.spec.ts
playwright.config.ts
worker/tests/
  auth.test.ts
  keys-*.test.ts
  scripts*.test.ts
...
```



Coverage policy

Item	Policy
Gate	95% lines on scoped core
Ratchet history	70 → 80 → 83 → 87 → 90 → 95
Include	plugins, storage (minus idb), store, results, sources, streams, data, chart pure helpers + series-factory/manager-access, worker auth/keys/runtime/scripts
Exclude from gate	drawing-layer.ts , pane-manager (unit elsewhere), runner chart apply, pyodide boot, legacy JS, Solid UI .tsx
Full report	bun test --coverage unscoped

Conventions

1. `import './setup'` first when tests touch store/plugins/storage.
2. **No real network** — mock `fetch` / `WebSocket` .
3. Table-driven tests for catalog built-ins.
4. Plugin fixtures in `tests/fixtures/plugins/` .
5. Prefer calling handlers with fake params over full browser for unit scope.

Playwright

Config (`playwright.config.ts`):

- `testDir: ./e2e`
- Chromium project
- `webServer: bun run build && bun run preview -- --host 127.0.0.1 --port 4173`
- `baseUrl: http://127.0.0.1:4173` (override `AXIS_E2E_BASE`)
- Timeout 60s; CI retries 1

Smoke mocks `/run` and `Binance`. Use `data-testid` attributes (`axis-btn-load` , `axis-manager` , ...).

Security suite

`bun run test:security` covers:

- Plugin URL scheme rejection
- Storage-via-URL rejection
- Poisoned `localStorage`
- Worker partition isolation
- If-Match 409
- Admin keys

Adding a unit test

1. Create `tests/<area>.test.ts` .



- 2. Import setup if needed.
- 3. Use fixtures under `tests/fixtures/`.
- 4. Run `bun run test:unit` and `bun run test:coverage:gate`.

CI linkage

See [CI and testing](#) for workflow job names and nightly schedule.

Failure modes

Failure	Mitigation
Coverage drop	Add tests in scoped files or deliberately exclude only with policy change
E2E flake	Mock network; single worker; stable testids
IDB in unit	Local storage falls back to memory maps

See also

- [CI and testing](#)
- [Plugin contracts](#)
- [Worker auth](#)

PLUGIN EXAMPLES

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED

THIS FILE IS PART OF PYNESCRYPT.

PYNESCRYPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY

IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY

THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR

(AT YOUR OPTION) ANY LATER VERSION.

PYNESCRYPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL, BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF

MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.

YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE

ALONG WITH PYNESCRYPT. IF NOT, SEE

[HTTPS://WWW.GNU.ORG/LICENSES/](https://www.gnu.org/licenses/).

SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Plugin examples" description: "Worked AXIS example plugins: CoinGecko source, Tiny Pine engine, Cloudflare DO stream — load paths and contracts."

PLUGIN EXAMPLES

Abstract

AXIS ships three **loadable example plugins** as ES modules under `public/plugins/` (also mirrored for reference under `src/plugins/example-*.js`). They are not built-ins; install via Manager → Plugins → **Load from URL**.

Full narrative: `src/plugins/README.md`.



Conceptual model

```
flowchart LR
    URL["/plugins/example-*.js"]
    URL --> LOADER[loadPluginFromUrl]
    LOADER --> REG[registry]
    REG --> PICKER[Source/Stream/Engine pickers]
```

Loading

1. Serve the PWA (Vite dev, preview, or `axis_pwa_server.py` on `dist`).
2. Open **Manager** → **Plugins**.
3. Paste a same-origin URL, for example:

```
http://127.0.0.1:8081/plugins/example-coingecko-source.js
http://127.0.0.1:3000/plugins/example-tiny-pyne-engine.js
http://127.0.0.1:8081/plugins/example-cf-do-stream.js
```

4. **Load** — registry updates; pickers show the new plugin.
 5. Persistence: `localStorage` `pynscript.axis.plugins.v1` restores on next visit.
- You may host modules on any origin that allows **module** CORS (same-origin is simplest).

Example catalog

File	id	kind	Purpose
<code>example-coingecko-source.js</code>	<code>coingecko</code>	source	Public CoinGecko market_chart → synthetic OHLC
<code>example-tiny-pyne-engine.js</code>	<code>tiny-pyne</code>	engine	In-browser JS DSL: sma/ema/rsi/plot/strategy
<code>example-cf-do-stream.js</code>	<code>cf-do</code>	stream	WS to Worker <code>/api/stream</code> SessionDO
External: <code>pyne-agent-worker</code> <code>/plugin/axis-pine-agent.js</code>	<code>pyne-agent</code>	component	NL → PYNE Agent chat (Cloudflare® Workers AI™); docs

Paths:

- Served: `public/plugins/`
- Source copies: `src/plugins/example-*.js`

CoinGecko source

Contract: `kind: 'source'`, `fetchHistorical`.

Config: `baseUrl` (default CoinGecko v3), `vsCurrency` (default `usd`).

Behavior:

- Maps common symbols (`BTC` → `bitcoin`, ...).
- Requests `market_chart` for a day window derived from interval × bar budget.



- Synthesizes OHLC from consecutive price points (API is not full candles).

Limits: public rate limits (~10–30 calls/min); CORS depends on CoinGecko policy.

Use when: demo custom source without a key.

Tiny Pine engine

Contract: `kind: 'engine', isReady, run.`

Behavior:

- Tokenizes a tiny expression language.
- Built-ins: `close`, `open`, `high`, `low`, `volume`, `sma`, `ema`, `rsi`, `plot`, `strategy.entry` / `strategy.close`.
- Returns `RunResult` with plots/events — **not** full PYNE fidelity.

Use when: offline demos without loading ~14MB Pyodide or hitting Flask.

Cloudflare DO stream

Contract: `kind: 'stream', start → stop.`

Config: `endpoint` — Worker base URL (`https://...` or `http://127.0.0.1:8787`). Empty → error.

URL built:

```
{endpoint as wss}/api/stream?session={symbol}@{interval}&symbol=...&interval=...
```

Requires: Worker with `SESSIONS` Durable Object bound and deployed ([durable objects](#)).

Use when: many tabs share one upstream Binance kline subscription.

Minimal authoring templates

Source

```
id: 'my-source',
name: 'My Source',
kind: 'source',
description: 'Shown in Manager',
configSchema: {
  baseUrl: { type: 'string', default: 'https://example.com', label: 'Base URL' },
},
async fetchHistorical({ symbol, interval, config }) {
  const res = await fetch(`${config.baseUrl}/bars?symbol=${symbol}&interval=${interval}`);
  return res.json(); // Bar[]
},
};
```



Stream

```
id: 'my-stream',
name: 'My Stream',
kind: 'stream',
start({ symbol, interval, onBar, onError, onStatus }) {
  const ws = new WebSocket('wss://example.com/stream');
  ws.onopen = () => onStatus({ state: 'open' });
  ws.onmessage = (ev) => {
    const b = JSON.parse(ev.data);
    onBar({ time: b.t, open: b.o, high: b.h, low: b.l, close: b.c, volume: b.v });
  };
  ws.onerror = () => onError(new Error('ws error'));
  return () => ws.close();
},
};
```

Engine

```
id: 'my-engine',
name: 'My Engine',
kind: 'engine',
async isReady() { return true; },
async run({ script, bars }) {
  return { status: 'success', plots: bars.map((b) => b.close), events: [], series: {}, meta: {} };
},
};
```

Storage plugins **cannot** be loaded from URL yet.

Invariants & edge cases

1. Treat plugin URLs as **executable code**.
2. `normalizePluginUrl` rewrites `/src/plugins/` → `/plugins/` for production.
3. Dangerous schemes rejected (`javascript:` , `vbscript:` , `data:text/html`).
4. After load, window may emit product-specific events (see README) for UI extensions.

Failure modes

Issue	Fix
Failed to restore URL	404 after deploy path change
CORS on CoinGecko	Offline mock or proxy
DO stream errors	Bind SessionDO; set endpoint
Tiny engine parse errors	Unsupported DSL constructs



See also

- [Dynamic loader](#)
- [Contracts](#)
- [Worker durable objects](#)